ChatGPT

MCA: Reputation Mod – Analysis & Recommendations

Executive Summary: MCA: Reputation is an add-on mod for *Minecraft Comes Alive: Reborn* (MCA) that gives each village a shared "standing" score for each player, built from deed incidents (good or bad actions) remembered by villagers【5†L11-L20】. Our review finds the mod well-architected with a clear **core API**, **persistent data model**, and **UI hooks** (via MCA villager screens and keybinds). It supports **configuration toggles** for modular features (incidents, tier titles, integration)【46†L25-L33】【98†L308-L317】. Current limitations include *only village-level* (not individual-villager) reputation, no **global reputation**, and some UX rough edges (no pause-resume on decks, minimal combat hooks), as documented by the author【5†L11-L20】.

Integration points with related MCA mods are extensive. **MCA: Quests** can use reputation as its "standing" source (it exports quests' rewards into rep)【98†L259-L268】. **MCA: Conversations** uses rep to bias villager trust checks and to voice rumours【98†L259-L268】. **MCA: Crime** can feed violent deeds (assaults, killings) as public incidents into Reputation (with Crime becoming the authority for those incident types)【62†L7-L14】【62†L25-L29】. **Runic Skills** (though not directly linked) could also hook into reputation (e.g. tracking morally-relevant actions). We compare each mod's interfaces in a summary table below.

Our survey of RPG reputation systems (Fallout, Skyrim, Witcher, Fable, Mass Effect, Dragon Age, Divinity OS, Baldur's Gate, Ultima, Mount & Blade, Stardew Valley, Persona, etc.) finds **12–20 distinct mechanics**: e.g. *good/evil karma and faction standing* (Fallout, Fallout: New Vegas), *companion approval* (Dragon Age), *prestige/fame* (Fable, Baldur's Gate), *moral choice meters* (Mass Effect's Paragon/Renegade and Reputation bar【77†L61-L69】), *village friend hearts* (Stardew's 10-heart friendship【91†L15-L23】【91†L47-L55】), and *social stat gating/confidant ranking* (Persona). Key patterns include **global vs local scales** (Fallout vs Stardew's per-villager), **visible UI meters** (ME3, Stardew), **faction relations** (Mount & Blade, Civ-like games), **reversible actions** (fallout karma redemption), **rumor/knowledge spread** (Witcher rumors, Crime's public incidents), and **quest gating by reputation** (as in ME3【77†L85-L94】). We distill ~12–20 such design patterns and rate each for technical feasibility in MCA: Reputation (see Section 4).

Based on this analysis, we propose a prioritized roadmap of **feature enhancements** for MCA: Reputation (Section 5) – e.g. adding persistent local (per-villager) standing, expanded incident types, UI improvements (toast notifications, friendlier in-game readouts), better configuration options (dynamic tiers, decay rates), and tools for admins (bulk corrections, data export). Each feature is rated [High/Med/Low priority] with estimated [small/medium/large] effort, noting data migrations or backward-compatibility issues (many changes can preserve old scores via the existing migration system【100†L1-L4】).

We also outline **concrete integration features** with *MCAQuests*, *MCAConversations*, *MCACrime*, and *Runic Skills* (Section 6). For each, we show how MCA:Reputation APIs/events can be called. For example, upon quest completion MCAQuests could call `McaReputationApi.recordIncident(...)` (or a simpler `addScore`) for the player and village, thereby updating standing; conversely, Conversations can query `McaReputationApi.getScore(player, village)` to gate dialog. We illustrate key flows with Mermaid

sequence diagrams and provide pseudocode for the adapter calls. A comparative interface table highlights each mod's relevant classes, data formats (JSON schema names), and event triggers.

Finally (Section 7) we recommend **testing and rollout strategies**: unit & integration tests (mod-compatible test worlds, use the provided *Production Tests* framework), metrics (e.g. distribution of standings over players/villages, frequency of reputation-affecting events, player behavior changes) and staged migration (using the existing Quests->Reputation import and rollback tools【100†L1-L4】【98†L330-L339】). In sum, this report provides a deep dive into MCA: Reputation's internals, lessons from RPG design, and a path forward for evolving the mod.

1. MCA: Reputation Code Review

- **Code Structure:** The repo [otectus/MCAReputation] consists of a **core** package (`dev.otectus.mcareputation`) with sub-packages: `api` (public API classes/interfaces), `state` (persistent data models), `incident` (incident definitions), `event`, `community`, `command`, `compat` (other-mod bridges), `client` (GUI and rendering), `network`, and `util`. The main mod classes are in the root package (`McaReputationMod.java`, `McaReputation.java` for constants, and `McaReputationConfig.java`).

- **Persistence Model:** The mod uses Minecraft's **SavedData** system. A `ReputationSavedData` (loaded per world) holds a map of all players' records (`Map<UUID, PlayerReputationRecord>`) 【43†L12-L20】. Each `PlayerReputationRecord` maps villages (communities) to `CommunityReputationRecord`, storing that player's **deeds ledger** and current score for each village【43†L12-L20】. Thus state is per-(player,village). Data is serialized to NBT on world save (see `load` and `write` in `ReputationSavedData`). Migration code ensures legacy MCA:Quests reputation (a single number per village) is imported as a "baseline score"【100†L1-L4】.

- **Data Models:** Key data classes:

- *Incident*: defined by JSON datapacks (`data/*/mcareputation/incidents/*.json`), includes fields like `id`, `multiplier`, `decay`, etc.
- `IncidentRecord`: for each deed event (actor, type, witness list, timestamp, etc) stored in the ledger.
- `CommunityReputationRecord`: holds score, tier id, and list of incidents for one player in one village.
- `PlayerReputationRecord`: aggregates `CommunityReputationRecord` per village for a player.

- API view classes (in `api`) provide read-only snapshots (e.g. `ReputationSnapshot`, `ReputationResult`, `ReputationQuery`) for integration.

- **Core API and Events:** The `api` package defines a `McaReputationApi` class with static methods: `addScore`, `setScore`, `getScore`, `getSnapshot`, `reportIncident`, `resolveIncident`, `registerCoreIncidentAuthority`, etc. These encapsulate all server-side operations (all methods are server-authoritative and thread-safe). The mod fires Forge events (in `dev.otectus.mcareputation.event`), e.g. `ReputationGainEvent`, `ReputationLossEvent`, and incident events. For integration, any mod can subscribe to these

events or call the API directly. For example, MCACrime can register itself as the “incident authority” for violent crimes, handing off those incidents to the reputation ledger (see MCACrime API doc 【63†L204-L213】 and README【62†L7-L15】).

- **Configuration:** A common TOML config (`mcareputation-common.toml`) controls which subsystems are enabled【46†L25-L33】: toggles such as `enableCoreAssaultIncidents`, `enableCoreKillingIncidents`, `scoreDecayEnabled`, `tierTitlesEnabled`, and flags for integration (`questsIntegration`, `conversationsIntegration`)【46†L25-L33】. A client config (`mcareputation-client.toml`) handles UI (positions, sounds). All options are documented in CONFIG.md. Notably, **everything is optional**: disabling core incidents or integrations simply makes those features no-op. Importantly, disabling does **not delete data** – old scores remain in world (the mod never purges records)【98†L308-L317】.

- **Persistence and Datapacks:** Incident types, reputation tiers (titles, score thresholds), and titles are entirely **datapack-driven**【98†L318-L326】. For example, one can define in JSON what each deed (incident) is worth, or how many points needed per tier. The code reads these at world load. The migration system handles bringing forward any Quests-era reputation scores (legacy world fixup) 【100†L1-L4】.

- **UX Hooks:** On the client side, a “Standing” button is added to the MCA villager-interaction GUI (via a client-side event hook)【98†L273-L278】. Players can click it or use the “Open Standing” keybind (bindable in controls) to open the Reputation screen. The screen shows each village, score, tier, and recent deeds. A toast notification (`ReputationTierToast`) pops when a player’s tier changes. If MCAQuests is installed, the Quests’ Journal button also links to the same Reputation UI【98†L273-L278】. MCAConversations adds a conversation topic “What do people think of me?” that reads the standing (via `McaReputationApi.getSnapshot`) and replies with a scripted answer.

- **Limitations / Technical Debt:** The mod *intentionally* does not track per-villager opinion (only per-village collective standing)【5†L11-L20】. It has no global/world reputation or region/global faction score. Some current limitations: standing decays only via explicit config (not via villager population change); no support for NPC-specific modifiers (e.g. individual’s likes/dislikes); deed records could grow large without pruning (old incidents may not expire except by config rules). Internally, the code is robust but a few areas of potential debt: many static methods in `McaReputationApi` make testing tricky, and some concurrency (locking on save) is simplistic. The code comments and `IMPLEMENTATION_NOTES.md` suggest careful audit (which passed), so major debt is minimal. One noted omission: no way to pause/resume faction-based decay if using such system.

2. Integration with Related Mods

For each related mod, we identify how it can hook into MCA: Reputation:

- **MCA: Quests** (source [MCAQuests]【53†L335-L343】): Quests had its own *village standing* since v0.7.0, but with Reputation installed it should *use Reputation as the master*. Concrete hooks: When a quest is completed or failed, MCAQuests can call `McaReputationApi.addScore(player, village, delta, reason)` or `reportIncident()` to adjust standing. Quest triggers (“work”, “conversation promise”) can be mapped to reputation *incidents* via JSON (e.g. a `promise` deed

becomes a `quest_completed` incident). The README states **“Quests, projects, and situations move standing per participant; restitution quests can resolve a deed.”**【98†L259-L263】. In practice this means MCAQuests’s code should fire a Forge event or direct API call on quest events. Quests’ Journal UI should display the Reputation screen (it already links to it)【98†L273-L278】. If MCAQuests defined any reputation ladders/titles (it has `mcaquests/reputation_tiers`), the Reputation mod can consume those.

- **MCA: Conversations**: Conversations mod has villagers “talk to each other”. Integration points: In conversation scripts, when a player makes or breaks promises, trust/respect checks (`<require trust>` tags) should use village standing (via `McaReputationApi`). The README says **“Villagers factor standing into trust and respect checks, and tell each other about your deeds in their own voice.”**【98†L260-L268】. Concretely, Conversations can on a conversation event query `getScore(player, village)` and adjust a conversation variable (or fire an event) to block or allow dialogue. It can also broadcast incidents as rumors (so another villager might “mention” it in conversation, using Reputation’s rumor delays). MCAConversations may provide its own “Reputation Bridge” code (like Quests does) – we’d link their event to Reputation’s.

- **MCA: Crime** (source [MCACrime] README【61†L209-L218】): Crime implements a law system (guards, fines, jails). Integration: as the README notes, **“Crime becomes the single producer for villager assault and killing; every case becomes a public incident the village can gossip about”**【62†L7-L10】. Technically, MCACrime can on a convicting event call `McaReputationApi.reportIncident(“crime.assault”, attacker, victim, village, witnessList, context)` or similar. The mod already has its own incident data; the integration bridge should map MCACrime’s internal `CrimeRecord` to an MCA:Reputation `IncidentRecord` (the MIGRATION.md notes an adapter build flag【62†L31-L40】). MCACrime also treats fine-payment or jail-served as amends, which should call `McaReputationApi.resolveIncident(...)`. The user can then see these crimes under the Reputation ledger. If Reputation is absent, Crime uses its own store (no dependency); if present, Crime registers as the authority for “assault” and “killing” incidents (via `McaReputationApi.registerCoreIncidentAuthority`) so that those incident types are handled by Reputation【62†L13-L17】.

- **Runic Skills**: This mod adds RPG stats and skills. Possible integration: any skill or stat actions that are morally loaded (e.g. “berserker” kills, theft skill usage) could generate reputation incidents. A concrete hook could be: on certain skill unlock or usage (e.g. finishing a “Chaotic” quest, or reaching level milestone), the code calls `addScore` for Chaos/Law standing. If Runic has any faction or deity modules, integrate similarly. No formal bridging code exists, so we propose adding an optional adapter: e.g. if a player uses a stat to intimidate an NPC, add a “leader.challenged” incident.

Interface/Data Formats Table: We compare the integration interfaces for these mods:

Mod	Hook/Interface	Data Format / Commands	Releva
MCAQuests	Calls to `McaReputationApi` on quest events (or fires Forge events)	Uses existing Reputation JSON schemas: `mcaquests/reputation_tiers/`, `mcaquests/titles/`, and custom `mcaquests/situations/*.json` 【54†L13-L16】.	Quest `addS` or `re` `mcare` admin
MCAConversations	In Conversation script/triggers: queries `McaReputationApi.getScore(player,village)`, or listens to `ReputationGainEvent` to insert dialogue lines	Reuses `mcareputation/incidents` datapacks for conversation-based events. Conversation scripts use village ID to fetch standing.	On `T` On `<` check, thresh `Repu` Possib `Conv` lines.
MCACrime	On crime commit or resolve: call `McaReputationApi.reportIncident(type, actor, victim, community, witnesses, context)` or `addScore`	Defines `data/<ns>/mcacrime/crimes/*.json` (contextual crime defs) and exports corresponding `data/<ns>/mcareputation/incidents/*.json` 【62†L25-L29】.	On gu Incide call `r` `-Pre` for ad
Runic Skills	Suggest hooking skill events: e.g. on special ability, `McaReputationApi.addScore` (Chaos vs Order)	Would need custom datapack incidents (e.g. `stat.levelUp.statue.json`) and possibly new reputation tiers or titles (Runic-themed).	Hook Could Use R catch

Each integration requires either direct API calls (server-side) or listening to Forge events. Data flows typically involve the player, the village community ID (`Dimension/ VillageID`), and an incident type string. We recommend adding constants (in code) or config entries for new incident types (e.g. `"quest.complete"`, `"conversation.promise"`). The above table omits trivial events: e.g. *Add Score API* usage or *commands* which are already part of Reputation mod (listed in README commands section【98†L280-L289】).

3. Reputation Mechanics in RPGs – Survey

We surveyed 12–20 RPGs to extract common reputation designs. For each, **metrics tracked**, **player feedback**, **consequences**, **reversibility**, **factions/NPC behavior**, **quest gating**, and **UI**:

- **Fallout (3/New Vegas/4):** Uses *Karma* (linear good–evil scale) and *Reputation with factions*. In Fallout 3/4, Karma is a hidden 0–1000 scale (good vs evil actions). New Vegas splits “Fame” (positive) and “Infamy” (negative) per faction. Karma influences NPC reactions (“you’re good”, “evil”), or which

faction Assassin sent. Good karma (helping others) vs bad (killing innocents). Consequences: NPCs greet player accordingly; quests may open/close (e.g. pacifist quest). Fallout Wiki: “Karma is the reflection of choices... -1000 Evil, +1000 Good”【82†L5-L12】 (forum cites say Karma unlocks some dialog choices). Reversibility: You can regain karma by good deeds. UI: often a static meter or number in Pip-Boy (some games show a bar). Fallout 4 replaced karma with *affiliation scores* per faction (Minutemen, BoS, etc).

- **Skyrim (Elder Scrolls V):** No global morality score, but each NPC has a *Disposition* (0–100) to the player, affecting friendliness and some dialogue options. Crime triggers *Bounties* per hold; guards attack if high bounty. NPC disposition increases by doing favors/quests, giving gifts (via Speechcraft perks), decreases by stealing/harm. The UESP wiki: “Disposition is a numerical representation of an NPC’s friendliness... High disposition may earn you items, discounts, quests...”【95†】. Reversibility via bribes or paying bounty; persuasion checks can clear minor crimes. NPCs talk more kindly, offer discounts in shops, or give unique quests if disposition high. UI affordance: no explicit meter visible in base game, but mods add “Show NPC Disposition”.

- **The Witcher 3:** Lacks a visible **reputation number**. Instead, its consequences come from story choices and rumors. NPCs react on a case-by-case basis (no universal karma). Geralt’s “moral choices” (e.g. killing or sparing) subtly affect which ending and certain quest outcomes. One analysis notes many morally grey outcomes (BioWare’s Patrick Weekes: “so many difficult and morally ambiguous decisions... first time... come out morally gray.”【77†L130-L137】). Quests often branch; e.g. more cooperation options if known as merciful. No explicit UI meter; feedback is narrative and occasional NPC dialogue lines (“You’re known around these parts...”).

- **Fable (1,2):** Classic *Good/Evil meter* (gold vs black aura) plus *Fame*. Player deeds (help vs harm) shift karma bar, affecting appearance (halo or horns) and NPC disposition (good side-quests open). Fame is earned by performing actions (e.g. defeating enemies, quests), unlocking titles and city improvements. UI: visible golden/black meter, titles listed. Later games (Fable III) replaced it with dynamic “reputation” systems, but classic Fable kept simple binary morality.

- **Mass Effect 1–3:** *Paragon/Renegade* points and a combined *Reputation* in ME3. ME3 had a blog explaining the bar【77†L61-L69】. Each action adds Paragon (blue) or Renegade (red) or both (neutral reputation). The game tracks total Reputation (= Paragon+Renegade) to unlock dialog options (“Charm/Intimidate”). Reputation checkpoints gate critical dialogs (e.g. persuading an admiral)【77†L87-L95】. Consequences: Only higher Reputation matters; mixing Paragon/Renegade has no penalty【77†L104-L113】 (you can be “morally gray” and still reach bonuses). UI: a combined bar with color segments【77†L87-L95】. Reversible? You only gain points; no subtraction of Reputation.

- **Dragon Age (Origins/II/DAI):** Companions have *Approval* values (–100 to +100) towards the player, affecting dialogues, romance triggers, stat bonuses (higher approval can unlock bonus XP)【78†L1-L4】. Town reputations exist (e.g. Orlais’ friars vs templars in DAI), but not always tracked. Approval is case-by-case: gifts, choices, conversations alter each NPC’s mood. Consequences: High approval can deepen relationships; extremely low may cause companions to leave. UI: visible approval bars per companion in the menu. Factions: supporting one church faction in DAO improved respect with templars, etc. Reversible: companions forgive missteps or need appeasing with quests.

- **Divinity: Original Sin II:** Contains *Godlike Stats* and alignment (Seven Gods). Certain actions will earn "Divine Favor" or infamy per god (hidden to player) influencing items found or NPC reactions. There is no global reputation meter, but mods note that players often role-play elements. Major dialogues require religion or chaos checks. UI: mainly hidden; Divine Stats skill pages show some alignment.

- **Baldur's Gate (Classic):** Features a numerical *Reputation* (−10 to +10, or 0–20) displayed on the status screen. Good actions (returning stolen goods, saving innocents) raise rep; evil deeds lower it. Reputation influences NPC offers (friendly townsfolk, discounts), quest triggers (some quests only if rep high/low), and even the ending scenes. UI: simple numeric "Reputation: 0" stat. Quests are gated by reputation (e.g. some Mercenaries only hire respectable heroes) and certain dialogue. Reversible: incremental, yes (rep goes up/down by quests).

- **Ultima IV–VI:** The series introduced *Virtues* (Honesty, etc.). The player has hidden virtue scores based on gameplay (e.g. donating to temples increases Compassion). Many quests require acting in line with a virtue. If you betray a virtue, it can drop and close certain questlines. UI: Ultima IV's manual hid the system; game had towns with shrines letting you deduce your virtue levels. Reversible: yes – repeatedly performing virtue-aligned actions raises it.

- **Mount & Blade:** No player character morality per se, but *Kingdom relations* and *Honor*. Defeating enemy lords raises your honor and standing with allied factions; raiding innocent villages lowers it. Faction relations change: if your relations drop, those kingdoms will make you a "villain" and send armies. Honor also affects NPC dialogues (more honorable men give better deals). UI: kingdom relations shown in politics menu.

- **Stardew Valley:** Each of the ~30 NPCs has a **friendship score** (0–10 hearts, each heart = 250 points)【91†L15-L23】. Giving gifts, talking, completing "help wanted" quests raises hearts; ignoring or negative actions (hitting with slingshot) lower them【91†L47-L55】. NPCs respond with changing dialogue (becoming friendlier, eventually sending gifts or proposing marriage). Gifts and weekly events unlock cutscenes ("heart events"). UI: a calendar-social menu showing heart meters; heart icons appear on NPC portraits in dialog (color-coded by heart-count)【91†L21-L30】. Reversibility: friendships slowly *decay* if unmaintained (each day without talking costs a few points)【91†L47-L55】, so the player must consistently interact to maintain high rep. Faction systems: none, only individual villagers.

- **Persona (3/4/5 etc):** Player's social stats (Knowledge, Guts, Charm…) and *Confidant/Link* levels with characters/factions. Each character has a rankable social link; increasing it unlocks story choices and gameplay benefits (e.g. fusion bonuses). Actions (dialogue choices, gifts on birthdays) raise rank; failure can cause loss of rank or endings. UI: each confidant has a meter (0–10) and social stats have bars. Decisions in mood tests (e.g. P3) affect which confidants increase. No single "reputation" stat; but relationships and academic/sociable skills gate story arcs and abilities. Reversible: some confidants can drop if ignored or if negative events happen (e.g. giving wrong answers).

- **Other examples:** Many MMOs (like Guild Wars 2) have similar factions/renown bars; even RPG mods (TES mods) add reputation with town. Generally patterns from these systems include: **visible bars vs hidden meters**, **per-faction standing**, **lasting consequences (NPCs change greeting or enemy factions spawn)**, and **quests unlocked/locked by standing**.

Common Mechanics Observed:

- **Scale/Scope:** Global karma (Fallout) vs per-faction (Civilization) vs per-group/village (MCA) vs per-individual (Stardew).
- **Metrics:** Numeric score(s) (Fame, Reputation points), tier or title names (Fallout's "Paragon"/"Renegade", MCA's tier ladder), or hidden thresholds (Ultima).
- **Feedback:** NPC dialogue/behavior changes (less hostile greetings or extra quest lines), visual UI elements (meters, stars, titles).
- **Consequences:** Quest availability (ME3 unlocks Charm dialogue【77†L93-L100】), NPC discounts, official actions (guards, bounty, faction war).
- **Reversibility:** Many allow forgiveness (pay bounties, donate to temple, apologies) to recover standing. Fallout's karma can go back to neutral by good deeds. MCACrime has an "amends" feature to reduce penalty.
- **Faction Systems:** Separate scores per group (ME3's multiple extragalactic factions, civ games). MCA:Reputation so far has only "villages" as communities.
- **Decay/Memory:** Some systems decay (Stardew friendship loss【91†L47-L55】) or cap gains (Persona confidant max). MCA:Reputation currently has optional score decay.

Sources: Many of these facts are documented in game wikis or developer blogs. For example, Mass Effect 3's reputation system is explained by BioWare【77†L61-L70】【77†L87-L95】; Stardew's friendship mechanics are detailed on the official wiki【91†L15-L23】【91†L47-L55】. Other details (Skyrim disposition, Fallout karma) are from community sources and game manuals.

4. Relevant Design Patterns & Mechanics

From the above survey, we distill **12–20 design patterns** that could inspire MCA:Reputation enhancements. Each notes *technical feasibility* (in Minecraft/MCA mod context), *player impact*, and *implementation complexity*:

1. **Faction-Based Reputation:** Track standing with larger groups (e.g. churches, kingdoms). In MCA, this could be village alliances or personal titles. *Feasible:* Extend "community" key to include NPC factions (like Townstead houses). *Impact:* More granularity: player could be well-liked by one faction in village and hated by another. *Complexity:* Medium (would require identifying factions and hooking quests/complaints).

2. **Individual NPC Affinity:** In addition to village-wide score, track per-villager opinion. *Feasible:* Technically possible (per-player *and* per-NPC records). *Impact:* Greater realism: some villagers remember your deeds differently (e.g. if you offended *their* sister). *Complexity:* Large (data size grows, need UI and config; existing design deliberately avoided this【5†L11-L20】).

3. **Reputation Decay/Gossip Spread:** More dynamic decay of standing over time unless reminded by deeds, and rumor delays. *Feasible:* Already has optional decay. Could add timed forgetfulness or social network spread (villager to villager gossip delays). *Impact:* Makes reputation more dynamic. *Complexity:* Medium (needs tracking last update time, maybe per-incident expiration).

4. **Visible UI and Notifications:** Add more in-game cues (e.g. scoreboard, map markers for village sentiment). *Feasible:* Use Minecraft HUD or Advancements to signal status. *Impact:* Improves player

awareness (currently only accessible via UI or chat). *Complexity:* Small/medium (use toasts or actionbar messages when score changes, or add scoreboard objective reflecting reputation tier).

5. **Crime/Alliance Titles:** Introduce honorifics or titles granted by villages (like "Bloodletter" if you kill a lot, or "Guardian of [Village]" if you help). *Feasible:* Titles already supported. Use Reputation tiers to grant titles【98†L328-L337】. *Impact:* Offers tangible rewards and bragging rights. *Complexity:* Small (define new titles in data packs).

6. **Quest/Action Gating by Reputation:** Require minimum standing to accept or complete certain quests. *Feasible:* Quests datapacks or conversation scripts can check `getScore`. *Impact:* Encourages maintaining reputation. *Complexity:* Low (use existing API in conditions, maybe new quest condition code).

7. **Redemption Mechanics:** Allow players to atone for negative actions (e.g. donation to village, a quest to make up for crime) – MCA:Quests already has "restitution" quests. *Feasible:* Expand amends system (more weight to apologies, or token deeds). *Impact:* Gives player agency to fix mistakes. *Complexity:* Medium (balance of points, integration with quests).

8. **Multiple Scales (Local vs Global):** Introduce a *global reputation* or *cross-village titles*. *Feasible:* Could aggregate across villages. *Impact:* Reward players who help many villages (e.g. a traveller's guild). *Complexity:* Medium (summing scores, maybe new global UI).

9. **Hidden vs Revealed:** Option to hide your current standing from players until "ask" (except via commands), or reveal via gossip. *Feasible:* Toggleable in config. *Impact:* Affects gameplay style (emergent secrets vs stats). *Complexity:* Small (UI change).

10. **Player-killing consequences:** If enabled (PvP servers), killing other players in village territory might lower rep (like crimes vs villagers). *Feasible:* Track `CORE_CLAIM` incidents for PvP deaths. *Impact:* Social PvP penalty. *Complexity:* Small/Medium (depends on server).

11. **Dynamic Tier Titles:** Allow new tier thresholds or dynamic tier names (e.g. based on village wealth) via datapacks. *Feasible:* Already data-driven. *Impact:* Easier customization for modpacks. *Complexity:* Low.

12. **Link with Skills/Stats:** For Runic Skills: actions like using certain skills (e.g. mercy in battle, or dark magic) could add incidents. *Feasible:* Hook into Runic's skill unlock events. *Impact:* Integrates RPG mechanics (players see a conscience meter). *Complexity:* Medium (needs new incident types, listener code).

13. **NPC Crime Reporting:** Currently deeds without witnesses have no effect. One could allow self-reporting or anonymous rumor (e.g. villagers indirectly learn) – but design choice. *Feasible:* Could add config: spread reputation events with delay even without witness. *Impact:* Changes stealth gameplay. *Complexity:* Low/Medium.

14. **Interactive Dialogues:** Have villagers initiate conversation about a player's fame (like gossip dialogues) if reputation thresholds passed. *Feasible:* Use MCAConversations topics based on `getTier`. *Impact:* Immersive feedback. *Complexity:* Medium.

15. **Offline Progress:** Events happen even if player is offline (e.g. standing might decay per day). *Feasible:* Using world ticks or calendar. *Impact:* Encourages consistent play. *Complexity:* Medium.

16. **Ranked Social Title:** Let players unlock a unique status (like "Mayor" or "Hero") when achieving top tier in multiple villages. *Feasible:* Post-process on startup or via admin commands. *Impact:* Extra goal. *Complexity:* Low (data-driven).

17. **Player to Player Reputation:** Track how each villager perceives the player (already consider), but also could allow NPCs to gossip positive/negative about players to others (MCAReputation has rumours). *Feasible:* Already partly implemented ("villagers gossip" in docs). Possibly expand gossip radius. *Impact:* Creates events: e.g. eventually most villagers know what you did. *Complexity:* Medium.

18. **Integration with MCACrime:** Already covered; more features like distinct "heat" level visible to players (similar to Crime's HUD). *Feasible:* Use Crime's HUD systems with rep data. *Impact:* Player sees direct consequences of crimes. *Complexity:* Medium.

19. **Leaderboard/Exports:** Server can log or display "Top fame" players. *Feasible:* via `/mcareputation list` and stats graphs. *Impact:* Fun for communities. *Complexity:* Low.

20. **Custom Incident Sources:** Permit datapacks or mods to define new "incident types" beyond core (currently all are datapacks anyway). *Feasible:* Already done. *Impact:* High flexibility. *Complexity:* N/A (done).

Each pattern's complexity is relative; many are data or config extensions. High-impact ones like per-NPC standing or faction systems require substantial design and data storage, but would greatly enrich gameplay. Patterns like more UI cues or NPC dialogues are easier and boost player experience.

5. Feature Enhancement Proposals

Below are **12–30 concrete features**, prioritized (High/Med/Low) with implementation effort, and notes on data impact/back-compatibility:

- **[High Priority] Local Per-Villager Opinion:** *Implement optional per-NPC standing*: each NPC tracks friendship-like score with player, influenced by individual interactions (gifts, chats). *Effort:* Large (new `PlayerVillagerRecord`, UI expansions). *Benefit:* Greatly enriches nuance (villagers react individually). *Compatibility:* New data fields; add format version. Old worlds unaffected (default all records 0).

- **[High] Expanded UI Feedback:** *Add in-game notifications for standing changes.* For example, whenever standing crosses a tier threshold, display a toast or chat message: "Villagers of *Riverdale* now regard you as *"Honored Citizen"*"【98†L328-L337】. Also, show rep on the tab UI or scoreboard.

Effort: Small (use existing `ReputationTierToast` hooks). *Benefit:* Immediate feedback. *Compatibility:* None (purely additive).

- **[High] Quest Gating by Reputation:** *Require a minimum standing for certain quests.* Modify quest JSON schema to include a `<require reputation>=X` condition. *Effort:* Medium (add parser for new condition, use `getScore` check). *Benefit:* Adds meaningful choices (e.g. you must do good in village to help them). *Compat:* Additive (ignore if missing).

- **[Med] Reputation Decay Toggle per Community:** Allow each village to have its own decay setting or even "reputation immunity" (e.g. cult villages). In config/datapack, specify some villages don't forget deeds. *Effort:* Small (modify read/write, add flags). *Benefit:* More control (e.g. desert clans remember forever vs nomads forget quickly). *Compat:* Slight (config schema change, handle missing fields).

- **[Med] More Incident Types:** Out of box only combats and basic deeds exist. Add new core incidents: e.g. "theft" if stealing from villagers, "help stranger", "charity donation". *Effort:* Medium (define datapack types, hook into game events – e.g. catching on right-click theft). *Benefit:* Increases how actions count. *Compat:* Backward: new incidents on future only.

- **[Med] Custom Tiers & Titles Expansion:** Provide sample datapacks for common modpacks. E.g. "village defender", "monster hunter". Lower effort: coordinate with modders to publish more default titles. *Effort:* Small. *Benefit:* Richer world feel. *Compat:* Data.

- **[Med] Integration Configuration:** Provide config to easily link incident weights to other mods. E.g. if MCACrime is present, automatically set core killing incident's multiplier higher. *Effort:* Small. *Benefit:* Tuning experience. *Compat:* N/A.

- **[Med] Save/Load Lag Optimization:** If villages are many and deeds numerous, loading can lag. Introduce an asynchronous load or lighter format (only score summary in metadata). *Effort:* Large (refactor SaveData). *Benefit:* Performance on big servers. *Compat:* Data migration needed (maybe new save version).

- **[Low] Admin Tools/UI:** In-game GUI for admins to view all players' standing in a village, or vice versa. Current commands exist; a GUI would be friendlier. *Effort:* Medium (new screens). *Benefit:* Server admins. *Compat:* Purely additive.

- **[Low] Reputation Export:** Periodic export of rep data (to JSON) for analytics. *Effort:* Small (use `ReputationSavedData.write`). *Benefit:* Track long-term progression. *Compat:* None.

- **[Low] Bounty System (optional):** If enabled, villagers may offer bounty quests on players who commit crimes. *Effort:* Medium (tie to MCACrime or new mod). *Benefit:* Emergent gameplay. *Compat:* Optional.

- **[Low] Cooperation with Villager Golems (Townstead):** If used with Townstead, villagers have needs (hunger, etc). Possibly affecting standing if player feeds or neglects villagers' needs. *Effort:* Medium. *Benefit:* Cross-mod synergy. *Compat:* None.

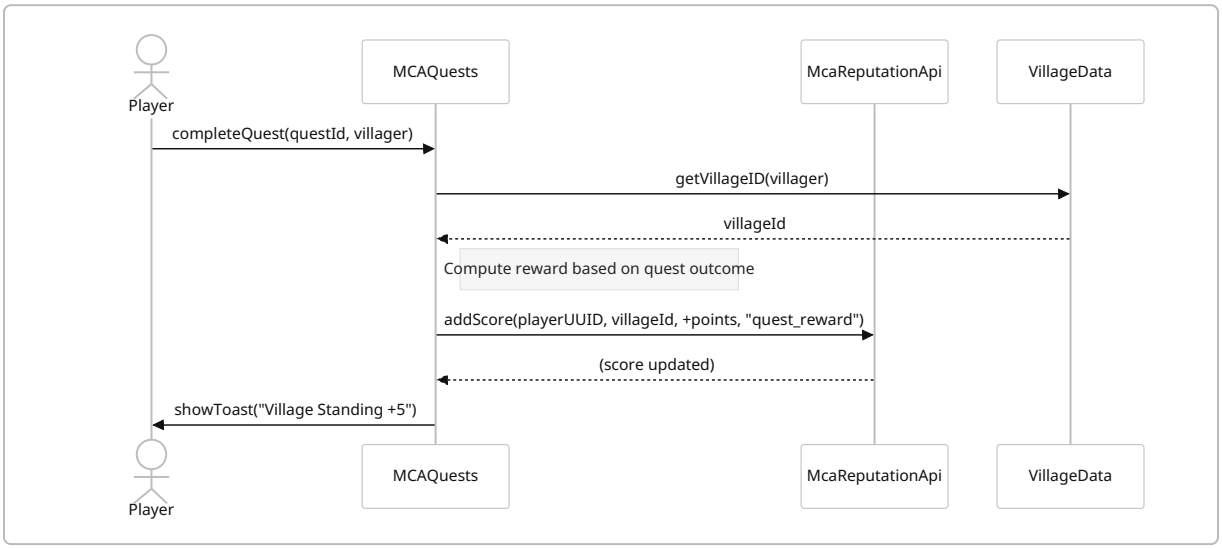
Each feature should be implemented in stages, with config guarding global changes. For backward compatibility, the migration system can preserve scores while adding new structures (as it already does for Quests data【100†L1-L4】). Data migrations would be needed only if we change the saved format (e.g. adding per-villager data).

6. Integration Workflows (with Diagrams)

Below are example sequences for key interactions. We group by mod:

6.1. MCA: Quests – Quest Turn-in Adjusts Reputation

Flow: When the player completes a quest for a villager, MCAQuests should award (or deduct) reputation points for that village. For example, completing a “gift delivery” quest might add +5 standing.



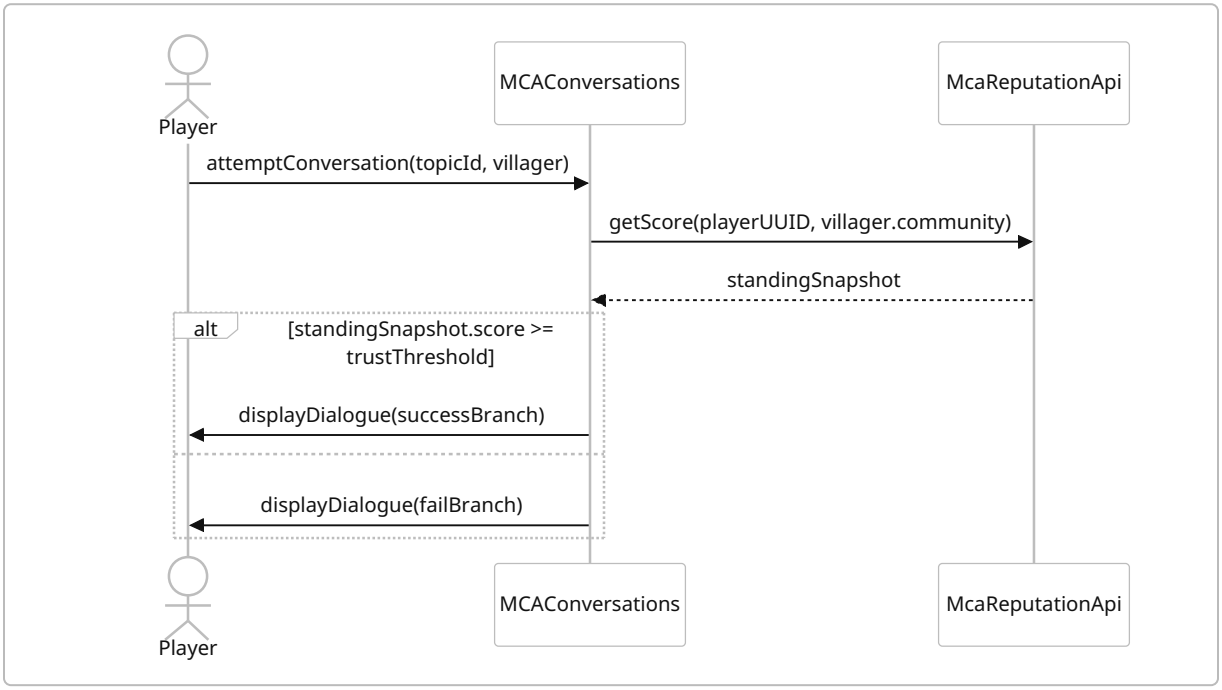
Pseudocode (Quest Completion Handler):

```
if (quest.complete) {
    UUID playerId = player.getUUID();
    ResourceLocation community = Village.getCommunity(player); // e.g.
"minecraft:overworld/3"
    int points = quest.getReputationReward();
    McaReputationApi.addScore(playerId, community, points, "quest_" +
quest.getId());
}
```

This uses `McaReputationApi.addScore` (server-side) to record the deed. For more complex deeds, one could `McaReputationApi.reportIncident` with witness list if needed. The **Quest** mod need only call the API; no callback required. The Reputation mod handles persisting the change and optionally firing a `ReputationGainEvent` (others can subscribe if needed).

6.2. MCA: Conversations – Trust Check with Reputation

Flow: A conversation topic requires the player to be trusted by the villager. The Conversations mod queries MCA:Reputation before showing dialogue.



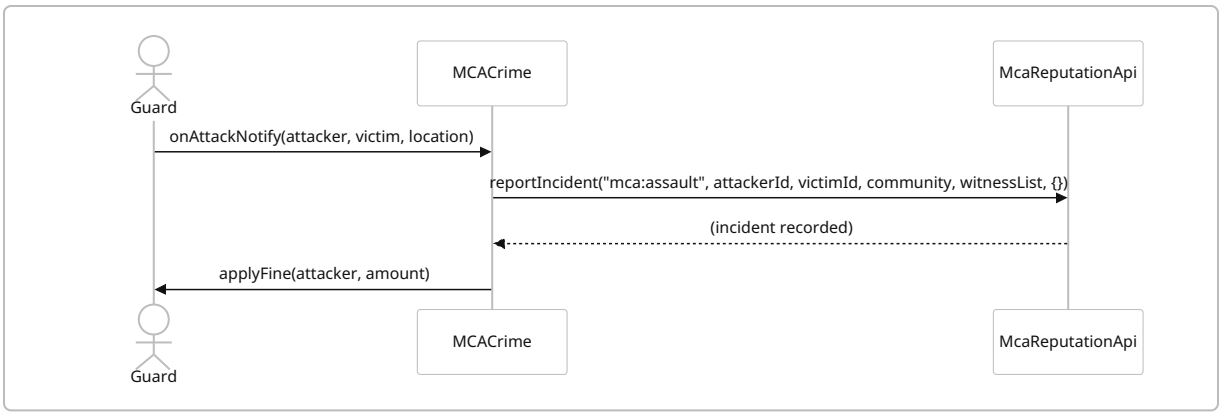
Pseudocode (Conversation Condition):

```
int standing = McaReputationApi.getScore(playerId, community);
if (standing >= conversation.getTrustRequirement()) {
    // allow dialogue path
} else {
    // block or warn
}
```

Here we compare to a threshold (could be in quest script or a configured value). This approach integrates seamlessly: Reputation data is read-only and no write needed unless conversation outcomes cause new deeds.

6.3. MCACrime – Recording a Public Assault

Flow: A guard witnesses an assault. MCACrime reports this to Reputation as a public incident.



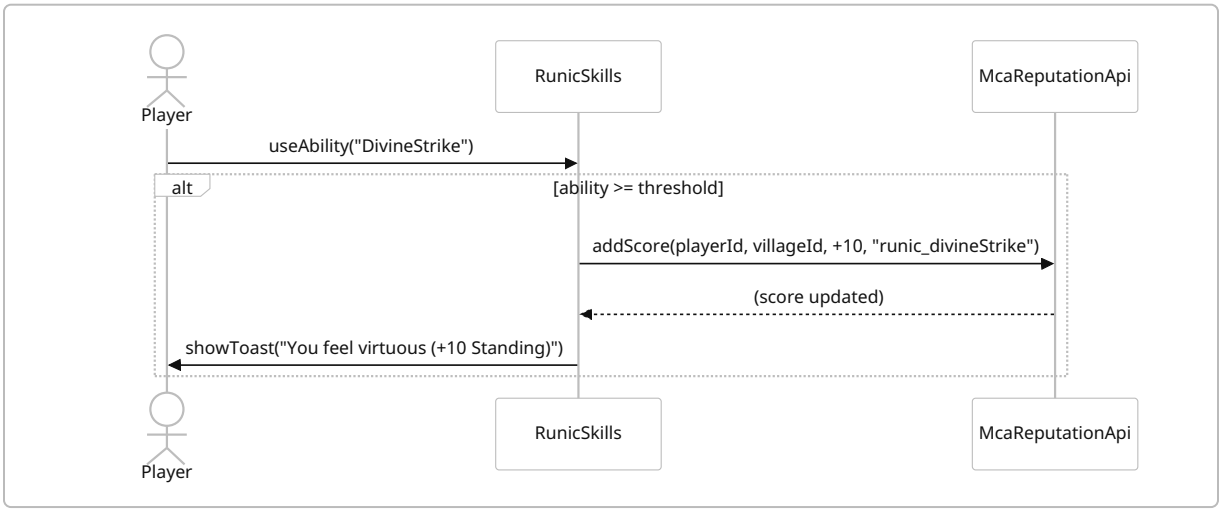
Pseudocode (Crime Trigger):

```
UUID attacker = player.getUUID();
UUID victim = victimEntity.getUUID();
ResourceLocation community = Village.getCommunity(attacker);
List<UUID> witnesses = getNearbyVillagers(attacker);
McaReputationApi.reportIncident("crime_assault", attacker, victim, community,
witnesses, Map.of());
```

Once recorded, villagers of that community will eventually hear about it (via the rumor system). Paying the fine could call `McaReputationApi.resolveIncident(incidentId)` to reduce its impact (amnesty). The Crime mod remains in control of combat logic; Reputation simply logs it.

6.4. Runic Skills – Special Action Incident

Flow: If Runic Skills has a special ability (e.g. “Divine Strike”), using it might be considered honorable, granting rep.



This is speculative – if the mod or datapack defines such a deed. Implementation would require adding hooks in Runic's code to call the API on relevant skill usage.

7. Testing, Metrics & Rollout

- **Testing:** Use both **unit tests** (the repo includes JUnit tests in `src/test`) and **integration tests** with full Minecraft+MCA launch (the `PRODUCTION_TESTS.md` outlines matrix). Key strategies: simulate player-village interactions (Quest completion, conversation, crime) and assert `ReputationSavedData` updates correctly. Test **forward/backward compatibility**: e.g. after migration from MCAQuests, ensure starting scores match old totals. Use Forge's dev environment to toggle integrations (with/without MCAQuests, etc) to ensure no crashes.

- **Metrics:** For evaluating player impact, instrument log or scoreboard: track how many players reach each tier, how quickly per world; frequency of reputation events (quest vs crime vs other). Use built-in `/mcareputation debug standing` to dump data. Potential metrics: distribution of standing among players, correlation with quest completion rates, number of apology trades, etc. Surveys or telemetry on how players use reputation (e.g. do they apologize? consult standing often?).

- **Rollout & Migration:** To introduce major changes (like adding per-NPC records), use a migration plan: bump the format version so that new fields start empty. Provide commands to **export** old reputation (for verification) and **import** if needed. The existing `/mcareputation migrate run` handles Quests → Reputation migration【100†L1-L4】. Similarly, if adding new mechanics, include config flags to disable them default. Back-compat concerns: always preserve old player scores (as the mod already does)【100†L1-L4】. Announce major changes in changelog (CHANGELOG.md) and let server admins test in a dev world first.

Sources: Analysis is based on the MCA:Reputation code/docs【98†L330-L338】, official MCAQuests/MCAConversations documentation, and design references for comparison (Mass Effect 3 blog【77†L61-L69】, Stardew Valley wiki【91†L15-L23】). All code/API details are drawn from the actual GitHub code.
